

FutureKawa

Documentation Frontend — Siège

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 05/07/2026

Projet : FutureKawa

Sommaire

Frontend Siège — FutureKawa

01 — Vue d'ensemble

02 — Vue.js — les fondamentaux

03 — Architecture & design atomique

04 — Gestion d'état avec Pinia

05 — Services API & Tailwind

06 — Tests

Frontend Siège — FutureKawa

Projet	FutureKawa
Livrable	Documentation Frontend Siège
Périmètre	Interface web de supervision multi-plantations
Framework	Vue.js 3 + Vite + TypeScript
État	Pinia
Styles	Tailwind CSS + Remix Icon
Graphiques	Chart.js 4
Équipe	Rudy · Florian · Noé · Josselin

Sommaire

Section	Fichier
Vue d'ensemble	01-vue-ensemble.md
Vue.js — les fondamentaux	02-vuejs.md
Architecture & design atomique	03-architecture-atomic.md
Gestion d'état avec Pinia	04-pinia-state.md
Services API & Tailwind	05-services-tailwind.md
Tests	06-tests.md

Pile technique

Dépendance	Rôle
vue 3	Framework UI
vue-router 4	Navigation entre les pages

Dépendance	Rôle
<code>pinia</code>	Gestion d'état centralisée (auth, sites, users)
<code>axios</code>	Appels HTTP vers l'API siège
<code>chart.js</code> 4	Graphiques de mesures consolidées
<code>tailwindcss</code>	Styles utilitaires
<code>@remixicon/vue</code>	Icônes
<code>typescript</code> + <code>vue-tsc</code>	Typage strict de bout en bout
<code>vitest</code> + <code>@vue/test-utils</code>	Tests unitaires

Ce qui distingue ce front de l'exploitation

	Siège	Exploitation
État	Pinia	Store léger maison
Styles	Tailwind CSS	CSS + Font Awesome
Structure	Atomic design + <code>pages/</code>	<code>views/</code>
Temps réel	Non (lit le cache siège)	Oui (MQTT)
Chart.js	v4	v3

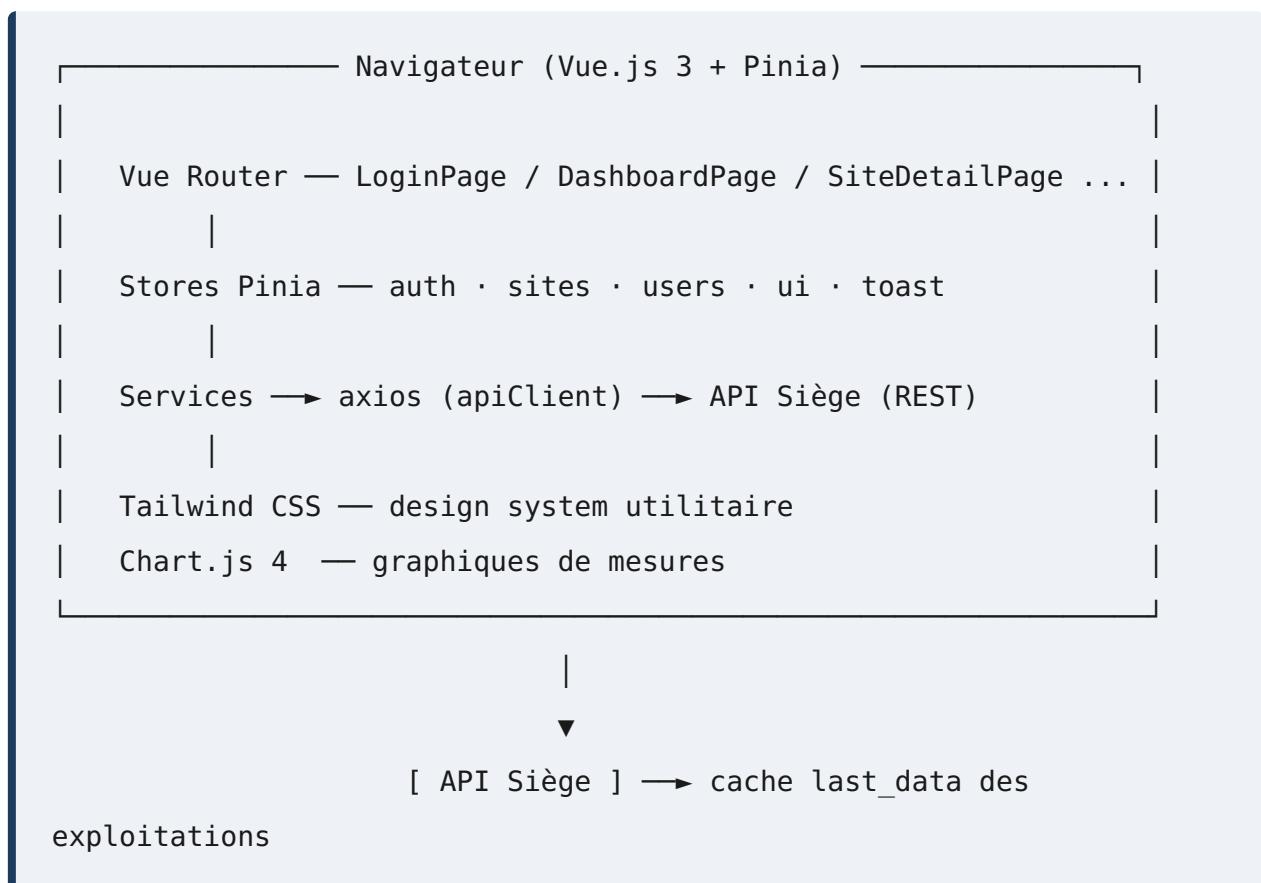
01 — Vue d'ensemble

Rôle du frontend siège

C'est l'interface de la **direction et des superviseurs**. Elle offre une vue consolidée de **toutes les plantations** : carte des sites, état de connexion, mesures agrégées, détail par site, lot ou capteur. Elle sert aussi à administrer les sites et les utilisateurs du siège.

Contrairement au front exploitation, elle **ne reçoit pas de MQTT** : elle lit les données déjà agrégées et mises en cache par le [backend siège](#).

Architecture d'ensemble



Les pages (vue-router)

Page	Rôle
LoginPage	Connexion

Page	Rôle
SetPasswordPage	Définition du mot de passe (invitation)
DashboardPage	Vue d'ensemble multi-sites
SiteDetailPage	Détail d'une plantation
SiteSectionPage	Section/zone d'un site
SiteChartPage	Graphiques d'un site
SiteLotDetailPage	Détail d'un lot d'un site
AdminSitesPage	Administration des sites
AdminUsersPage	Administration des utilisateurs
ProfilePage	Profil de l'utilisateur connecté

Trois choix structurants

Choix	Détail
Pinia	Gestion d'état centralisée (session, sites, utilisateurs, notifications)
Tailwind CSS	Styles utilitaires directement dans le template, design cohérent
Atomic design	Composants classés en <code>atoms</code> / <code>molecules</code> / <code>organisms</code>

Démarrage

```
cd front-siege
npm install
npm run serve      # serveur de développement Vite

npm run build      # build de production
npm run test       # tests unitaires (Vitest)
```

02 — Vue.js — les fondamentaux

Qu'est-ce que Vue.js ?

Vue.js est un framework JavaScript pour construire des interfaces web dynamiques. On décrit **ce qu'on veut afficher** en fonction de données ; Vue met à jour l'écran **automatiquement** quand ces données changent.

Le front siège utilise **Vue 3**, la **Composition API** (`<script setup>`), **Vue Router** pour la navigation et **Pinia** pour l'état — voir [Gestion d'état avec Pinia](#).

Une présentation détaillée des directives et du cycle de vie figure dans le [livrable Frontend Exploitation](#). Cette section rappelle l'essentiel et met l'accent sur les usages propres au siège.

Le composant à fichier unique (`.vue`)

Chaque composant regroupe structure, logique et style :

```
<template>
  <div class="rounded-lg p-4 shadow">      <!-- classes Tailwind -->
    <h3 class="font-semibold">{{ site.name }}</h3>
    <span :class="statusColor">{{ site.status }}</span>
  </div>
</template>

<script setup lang="ts">
import { computed } from 'vue'
const props = defineProps<{ site: Site }>()

const statusColor = computed(() =>
  props.site.status === 'active' ? 'text-green-600' : 'text-red-600'
```

```
)  
</script>
```

Ici, deux marques du siège apparaissent déjà : les **classes Tailwind** dans le template et le **typage TypeScript** des props.

La réactivité

Outil	Rôle
<code>ref()</code>	Valeur réactive simple
<code>computed()</code>	Valeur dérivée recalculée automatiquement
<code>watch()</code>	Réagir à un changement
<code>defineProps()</code>	Recevoir des données du parent (typées)
<code>defineEmits()</code>	Émettre un événement vers le parent

```
const sites = ref<Site[]>([])  
const nbActifs = computed(() => sites.value.filter(s => s.status ===  
'active').length)  
// nbActifs se met à jour tout seul dès que sites change
```

Composition API : `<script setup>`

Le siège utilise systématiquement la syntaxe `<script setup lang="ts">`, la plus concise de Vue 3 :

- Les imports, `ref`, `computed`, fonctions sont déclarés directement.
- Tout ce qui est déclaré est automatiquement disponible dans le `<template>`.
- Le typage TypeScript est natif (`vue-tsc` vérifie les templates au build).

```
<script setup lang="ts">  
import { useSitesStore } from '@stores/sites'  
const store = useSitesStore() // accès à l'état global (Pinia)
```



```
onMounted(() => store.fetchSites())  
</script>
```

Ce que Vue apporte, complété par l'écosystème siège

Brique	Rôle	Section
Vue 3	Réactivité, composants	—
Vue Router	Navigation SPA entre les pages	03
Pinia	État partagé (session, sites, users)	04
Tailwind CSS	Styles utilitaires	05
TypeScript / vue-tsc	Typage strict, vérification des templates	—

Pourquoi cette pile pour le siège ?

Besoin du siège	Réponse
Beaucoup d'état partagé (sites, session, notifications)	Pinia centralise
Interface d'administration soignée et cohérente	Tailwind + design atomique
Fiabilité sur des données complexes	TypeScript strict
Réutilisation de composants (cartes, stats, graphiques)	Composants Vue atomiques

03 — Architecture & design atomique

Arborescence du code source

```
front-siege/src/
├─ main.ts           # point d'entrée : crée l'app, branche Pinia
+ router
├─ App.vue          # composant racine
├─ router/
│   └─ index.ts      # routes (14 chemins) + garde
d'authentification
├─ pages/            # une page par route (Dashboard, SiteDetail,
Admin...)
├─ components/       # design atomique (atoms / molecules /
organisms)
├─ stores/           # état Pinia (auth, sites, users, ui, toast)
├─ services/         # apiClient axios + appels API
├─ utils/            # helpers (csv, ...)
├─ types/            # types TypeScript (auth, site)
└─ style.css         # entrée Tailwind
```

Le point d'entrée

```
// main.ts
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import router from './router'
import App from './App.vue'
import './style.css'

const app = createApp(App)
app.use(createPinia()) // gestion d'état
app.use(router)       // navigation
app.mount('#app')
```

Deux plugins sont branchés dès le départ : **Pinia** (état) et **Vue Router** (navigation).

Le design atomique

Les composants sont organisés selon la méthode **Atomic Design**, qui classe l'interface par niveau de complexité :

atoms/ (brique complet)	→	molecules/ (assemblage)	→	organisms/ (section)	→	pages/ (écran
Button		SearchField		SiteTable		AdminSitesPage
Badge		StatCard		NavbarComplète		DashboardPage
Input		ChartCard		SiteGrid		SiteDetailPage

Niveau	Définition	Exemples dans le projet
Atoms	Éléments indivisibles	boutons, badges, champs
Molecules	Petits groupes d'atoms	carte de stat, champ de recherche
Organisms	Blocs autonomes	navbar, tableau de sites, grille
Pages	Écran complet lié à une route	DashboardPage , SiteDetailPage

Avantages : **réutilisation** maximale, cohérence visuelle, et composants faciles à tester isolément.

Composants transverses notables

Composant	Rôle
AppNavbar	Barre de navigation principale
AppToast	Notifications éphémères (succès/ erreur)

Composant	Rôle
StatCard	Tuile de statistique du dashboard
MeasureChart	Graphique de mesures (Chart.js 4)
LoadingSpinner	Indicateur de chargement
QrCode	Génération de QR codes

Le routeur et la garde d'authentification

L'application est une **SPA**. Le routeur protège les pages privées : si l'utilisateur n'est pas connecté, il est redirigé vers `LoginPage`.

```
// router/index.ts (schéma)
router.beforeEach((to) => {
  const auth = useAuthStore()
  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { name: 'login' }
  }
})
```

C'est le store Pinia `auth` qui détient l'état de connexion — voir [Gestion d'état avec Pinia](#).

Configuration Vite & TypeScript

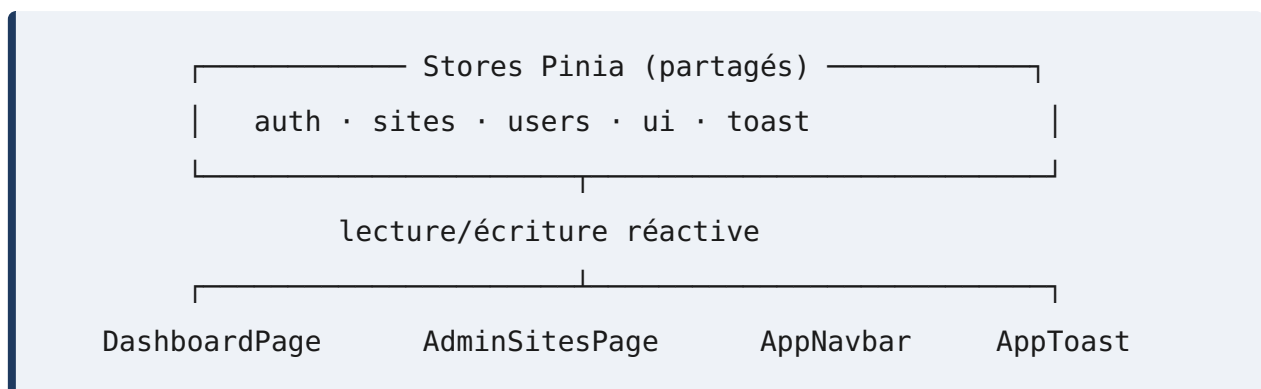
- `vite.config.ts` : alias d'import (`@/` , `@services/` , `@stores/`), proxy API, build.
- `vue-tsc` vérifie le typage **y compris dans les templates** au moment du build (`npm run build`).
- Résultat de production dans `dist/` , servi par Nginx en conteneur.

04 — Gestion d'état avec Pinia

Le problème que Pinia résout

Sans état centralisé, chaque composant recharge ses données et ne sait pas ce que font les autres. Qui est connecté ? La liste des sites est-elle à jour ? Comment afficher une notification depuis n'importe où ?

Pinia est la bibliothèque officielle de gestion d'état de Vue 3. Elle offre un **magasin partagé** (store) que tous les composants peuvent lire et modifier, avec la réactivité de Vue.



Les stores du projet

Store	Rôle
auth	Utilisateur connecté, token, rôle (admin/viewer), login/logout
sites	Liste des plantations, synchronisation, site sélectionné
users	Gestion des utilisateurs du siège
ui	État d'interface (menus, chargement)
toast	File des notifications éphémères

Anatomie d'un store (syntaxe « setup »)

Le projet utilise la syntaxe composition, la plus proche d'un composant :

```
// stores/auth.ts
export const useAuthStore = defineStore('auth', () => {
  // --- state ---
  const user = ref<User | null>(getStoredUser())
  const isLoading = ref(false)

  // --- getters (computed) ---
  const isAuthenticated = computed(() => user.value !== null)
  const isAdmin = computed(() => user.value?.role === 'admin')

  // --- actions ---
  const login = async (credentials: LoginCredentials) => {
    const response = await apiClient.post('/api/auth/login/',
credentials)
    const { token, user: userData } = response.data
    setToken(token)
    user.value = userData
  }

  return { user, isAuthenticated, isAdmin, login }
})
```

Élément	Équivalent Vue	Rôle
<code>ref()</code>	state	Les données du store
<code>computed()</code>	getters	Valeurs dérivées (<code>isAdmin</code>)
fonctions	actions	Logique métier (login, fetch)

Utilisation dans un composant

```
<script setup lang="ts">
import { useAuthStore } from '@stores/auth'
const auth = useAuthStore()

// lecture réactive
```

```

if (auth.isAdmin) { /* afficher le menu admin */ }

// action
await auth.login({ email, password })
</script>

```

N'importe quel composant appelant `useAuthStore()` obtient **la même instance** : l'état est réellement partagé.

Exemple concret : le store `sites`

```

AdminSitesPage — sites.fetchSites() → apiClient → GET /api/
sites/
      |
      ▼
    sites.list (ref) ← mis à jour
      |
DashboardPage et SiteGrid, abonnés, se rafraîchissent tout
seuls

```

Le store `sites` déclenche aussi la synchronisation (`sync`) d'un site, qui appelle côté backend l'endpoint proxy vers l'exploitation (voir [Backend Siège — Agrégation](#)).

Les notifications (`toast`)

Le store `toast` permet à **n'importe quel** composant d'afficher un message, capté par le composant global `AppToast` :

```

const toast = useToastStore()
toast.success('Site synchronisé')
toast.error('Échec de la connexion au site')

```

C'est un cas d'école de l'intérêt d'un état partagé : l'émetteur et l'afficheur sont totalement découplés.

Persistence de la session

Le store `auth` s'initialise à partir du `localStorage` (`getStoredUser()`), ce qui garde l'utilisateur connecté après un rafraîchissement de page. Le token est réinjecté dans axios (voir section suivante).

05 — Services API & Tailwind

Partie 1 — La communication avec l'API

Le client axios central

Comme le front exploitation, le siège isole la communication HTTP dans `src/services/`. Un client axios unique (`apiClient`) est configuré et réutilisé partout.

```
// services/apiClient.ts (schéma)
export const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
})

// le token de session est injecté sur chaque requête
export function setToken(token: string) { /* stocke + configure axios */ }
export function clearToken() { /* déconnexion */ }
```

Fonction	Rôle
<code>apiClient</code>	Instance axios partagée
<code>setToken</code> / <code>clearToken</code>	Gèrent le token d'auth (login/logout)
<code>getStoredUser</code> / <code>setStoredUser</code>	Persistent l'utilisateur en <code>localStorage</code>

Les services et les stores

Ici, la couche service est appelée **par les stores Pinia**, pas directement par les composants :

```
Composant → store Pinia (sites) → service (siteApi) → apiClient
→ API Siège
```

Service	Rôle
<code>apiClient</code>	Configuration axios + gestion du token
<code>siteApi</code>	Appels liés aux sites (liste, détail, sync, historiques)
<code>toast</code>	Utilitaire de notification

Correspondance avec le backend

Appel front	Endpoint backend
<code>authStore.login()</code>	<code>POST /api/auth/login/</code>
<code>sitesStore.fetchSites()</code>	<code>GET /api/sites/</code>
<code>sitesStore.sync(id)</code>	<code>POST /api/sites/{id}/sync/</code>
<code>usersStore.create()</code>	<code>POST /api/users/</code>

Voir le livrable [Backend Siège — Endpoints](#).

Partie 2 — Les styles avec Tailwind CSS

Qu'est-ce que Tailwind ?

Tailwind CSS est un framework de styles **utilitaires** : au lieu d'écrire du CSS séparé, on compose l'apparence directement dans le HTML avec de petites classes.

```

<!-- sans Tailwind : CSS séparé + nom de classe à inventer -->
<div class="carte">...</div>

<!-- avec Tailwind : tout est dans le template -->
<div class="rounded-lg bg-white p-4 shadow hover:shadow-md">...</div>

```

Classe	Effet
<code>p-4</code>	padding
<code>rounded-lg</code>	coins arrondis

Classe	Effet
<code>bg-white</code>	fond blanc
<code>text-green-600</code>	couleur de texte
<code>flex gap-2</code>	disposition flexbox
<code>hover:shadow-md</code>	ombre au survol

Pourquoi Tailwind pour le siège ?

Atout	Bénéfice
Cohérence	Même échelle d'espacements/couleurs partout
Rapidité	Pas de fichiers CSS à nommer et maintenir
Design system	Palette et typographie centralisées dans <code>tailwind.config.js</code>
Purge automatique	Seules les classes utilisées finissent dans le build

Configuration

- `tailwind.config.js` : couleurs, polices, thème du design system.
- `postcss.config.js` + `autoprefixer` : traitement CSS.
- `src/style.css` : import des couches Tailwind (`@tailwind base/`
`components/utilities`).

Icônes : Remix Icon

Les icônes viennent de `@remixicon/vue` , importées comme composants Vue :

```
<RiDashboardLine class="w-5 h-5" />
```

En résumé

Couche	Outil	Rôle
Données	axios (<code>apiClient</code>)	Appels REST vers l'API siège
État	Pinia	Partage et cache côté client
Styles	Tailwind CSS	Apparence cohérente
Icônes	Remix Icon	Pictogrammes

06 — Tests

Approche

Le front siège est testé au niveau **unitaire / composant** avec **Vitest** et `@vue/test-utils`, dans un environnement DOM simulé (`jsdom`). Le typage strict (`vue-tsc`) fournit une première couche de sûreté avant même l'exécution.

Ce qui est couvert

Domaine	Exemples
Composants	Rendu des atoms/molecules (StatCard, badges), états conditionnels
Stores Pinia	<code>auth</code> (login/logout, isAdmin), <code>sites</code> (fetch, sync), <code>toast</code>
Services	<code>apiClient</code> (injection du token), <code>siteApi</code>
Utils	Génération CSV (<code>utils/csv.js</code>)
Garde de route	Redirection si non authentifié

Les dossiers `__tests__` sont répartis au plus près du code (`components/`
`__tests__`, `stores/``__tests__`, `services/``__tests__`, `utils/``__tests__`).

Tester un store Pinia

```
import { setActivePinia, createPinia } from 'pinia'
import { useAuthStore } from '@stores/auth'

beforeEach(() => setActivePinia(createPinia()))

it('passe en admin après login', async () => {
  const auth = useAuthStore()
  vi.spyOn(apiClient, 'post').mockResolvedValue({
```

```
    data: { token: 'x', user: { role: 'admin' } },
  })
  await auth.login({ email: 'a@b.c', password: '...' })
  expect(auth.isAdmin).toBe(true)
})
```

Les appels réseau sont **mockés** : les tests ne dépendent d'aucune API.

Lancer les tests

```
cd front-siege
npm run test          # tests unitaires (Vitest)
npm run test:watch    # mode surveillance
npm run test:coverage # couverture
```

Un rapport JUnit (`junit.xml`) est produit pour l'intégration continue.

Intégration continue

Les tests tournent dans la pipeline GitLab CI (livrable **CI/CD**). La couverture détaillée est dans le livrable **Tests** (`07-couverture-front-siege.md`).